Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Team 56: OpenROAD VLSI Design 2

Bi-Weekly Update 4

Kevin, Talha, Githin, Giovani

Sponsor: Kevin Nowka

TA: Michael Johnson-Moore

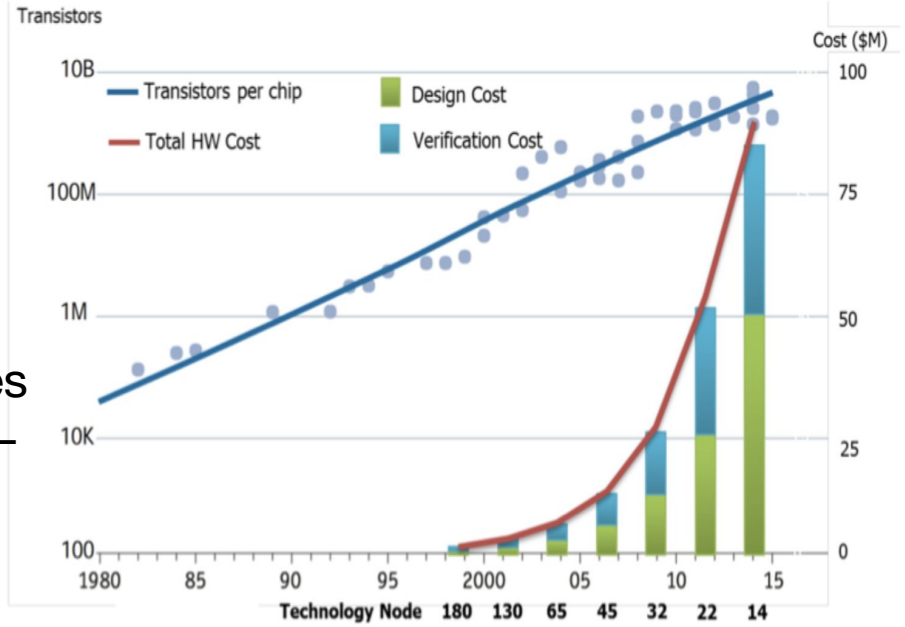
Project Summary

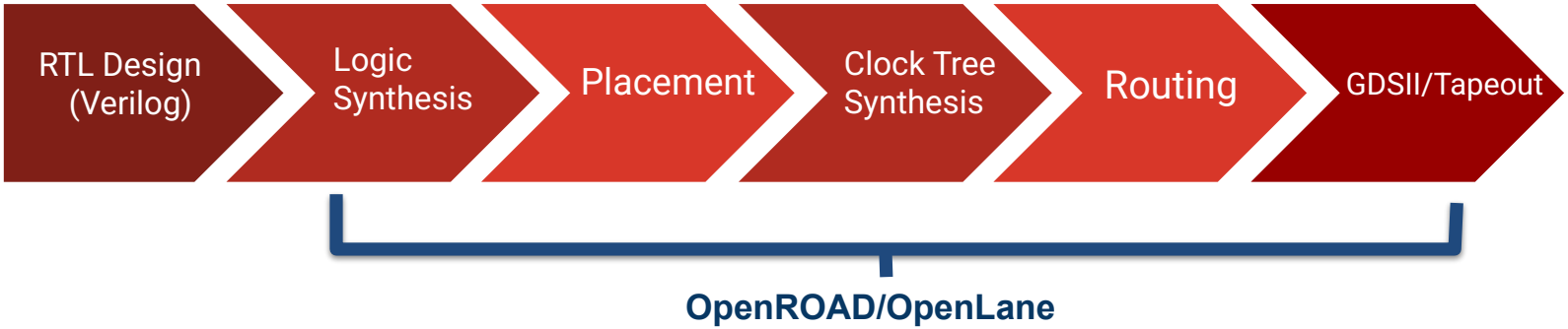
Problem statement:

- Traditional RTL-to-GDSII chip design is expensive, creating a cost barrier to developing a System on a Chip (SoC).

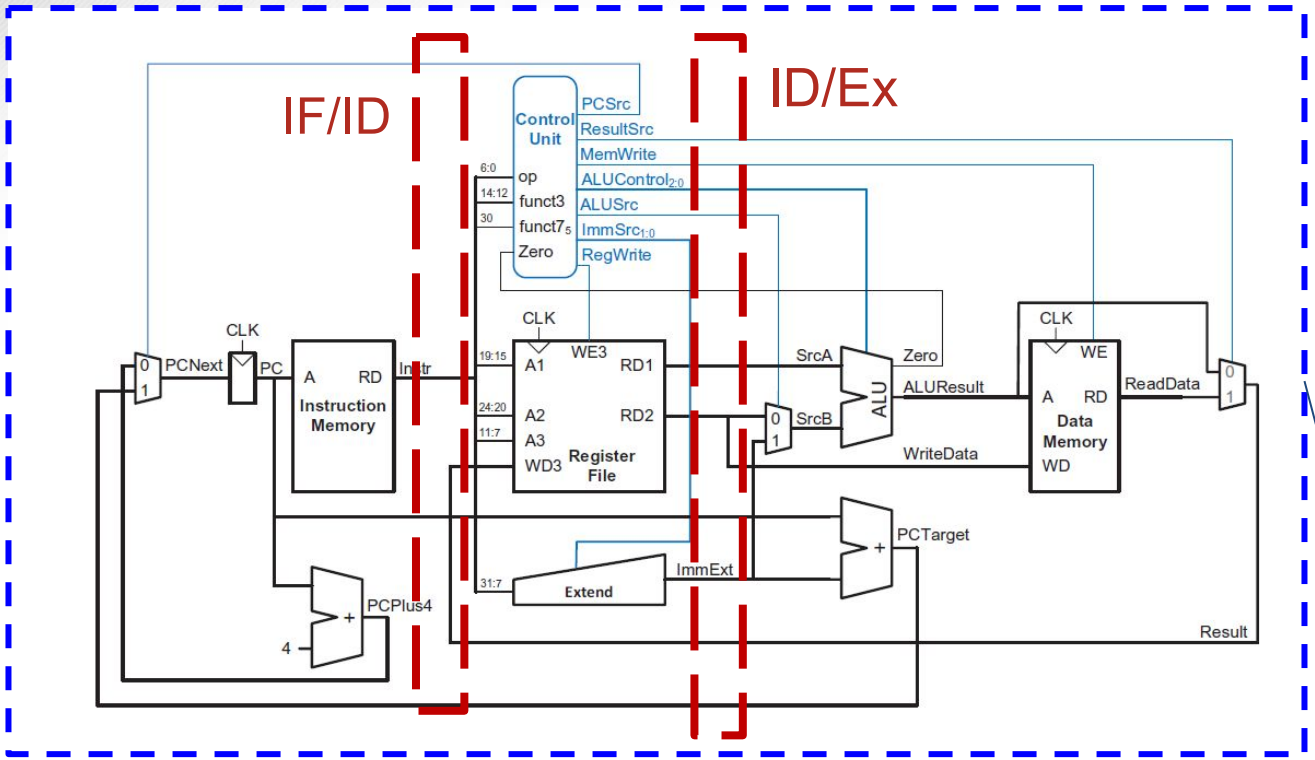
Our Solution:

- Create a 16-bit RISC-V using OpenROAD/OpenLane, which automates chip design by only needing only an RTL description to generate GDSII, significantly reducing cost.





System Overview



General Purpose Input Pins

General General Output Pins

Project Timeline (45 sec)

Finalize individual subsystem (completed 1/26)	Start integration/validation (Start by 2/2)	Implement pipeline stages & GPIO (Start by 2/16)	Validate pipeline stages & GPIO (Start by 3/2)	Start OpenLane/OpenROAD process (Start by 3/23)	Final Validation (Start by 4/6)

GPIO & OpenLane

Kevin Umanzor & Talha Ibrahim

Accomplishments since status update 3 11 hrs of effort	Ongoing progress/problems and plans until next presentation
• Implementation of Input/Output Pins including IO pad instantiation, ESD protection, and guard rings • SPI receiver + Boot FSM for serial instruction loading via uio pins	• Validating Input/Output pins (ESD protection, pad instantiation, guard rings) • OpenLane Config Files (config.json)

Dwight Look College of ENGINEERING
TEXAS A&M UNIVERSITY

GPIO & OpenLane

Kevin Umanzor & Talha Ibrahim

```
module tt_um_riscv16 (
    input  wire [7:0] ui_in,
    output wire [7:0] uo_out,
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,
    input  wire       rst_n
);
```

```
TEST 4: Byte splitting correctness
  ✅ PASS | ch0 uo_out  = 0x24 (low byte correct)
  ✅ PASS | ch0 uio_out = 0x00 (high byte correct)
  ✅ PASS | ch1 uo_out  = 0x00 (low byte correct)
  ✅ PASS | ch1 uio_out = 0x00 (high byte correct)
  ✅ PASS | ch2 uo_out  = 0x00 (low byte correct)
  ✅ PASS | ch2 uio_out = 0x00 (high byte correct)
  ✅ PASS | ch3 uo_out  = 0x06 (low byte correct)
  ✅ PASS | ch3 uio_out = 0x00 (high byte correct)
  ✅ PASS | ch4 uo_out  = 0x06 (low byte correct)
  ✅ PASS | ch4 uio_out = 0x00 (high byte correct)
  ✅ PASS | ch5 uo_out  = 0x06 (low byte correct)
  ✅ PASS | ch5 uio_out = 0x00 (high byte correct)
```

3 Stage Pipeline Integration

Giovani Giagnacovo & Githin Jonny

Accomplishments since status update 3 10 hrs of effort	Ongoing progress/problems and plans until next presentation
• Implemented 2 pipelined registers, one in between Instruction fetch and instruction decode and the other between instruction decode and instruction execute. • Implemented modules for basic hazard handling. • Resolved Branch logic bug	• Debug and update code so pipeline design is functional for all instruction types • Validate Pipeline stages

Jump/Branch Test Results

Giovani Giagnacovo & Githin Jonny

```
----------------------B-Type Testing-------------------
HEX   - imm    | Rs1 | Rs2 | func | OP
4144 - 01000 | 001 | 010 |  00  | 100 - BEQ x1, x2, 8
414c - 01000 | 001 | 010 |  01  | 100 - BNE x1, x2, 8
4154 - 01000 | 001 | 010 |  10  | 100 - BGT x1, x2, 8
415c - 01000 | 001 | 010 |  11  | 100 - BLT x1, x2, 8
```

```
Time=706000 | PC=0x0012 | Instr=0x4154 | ALU=0x002e | x1=0x0030 | x2=0x0002 | x3=0x0002 | zero=0
STORE: addr=002e data=0002 | MEM snapshot: [0]=xxxx [1]=xxxx [2]=0002


Time=716000 | PC=0x0014 | Instr=0x0000 | ALU=0x0000 | x1=0x0030 | x2=0x0002 | x3=0x0002 | zero=1
STORE: addr=0000 data=0000 | MEM snapshot: [0]=xxxx [1]=xxxx [2]=0002
```

```
Time=106000 | PC=0x0012 | Instr=0x415c | ALU=0x0002 | x1=0x0004 | x2=0x0002 | x3=0x0002 | zero=0
STORE: addr=0002 data=0002 | MEM snapshot: [0]=xxxx [1]=xxxx [2]=0002


Time=116000 | PC=0x0022 | Instr=0x0000 | ALU=0x0000 | x1=0x0004 | x2=0x0002 | x3=0x0002 | zero=1
STORE: addr=0000 data=0000 | MEM snapshot: [0]=xxxx [1]=xxxx [2]=0002
```

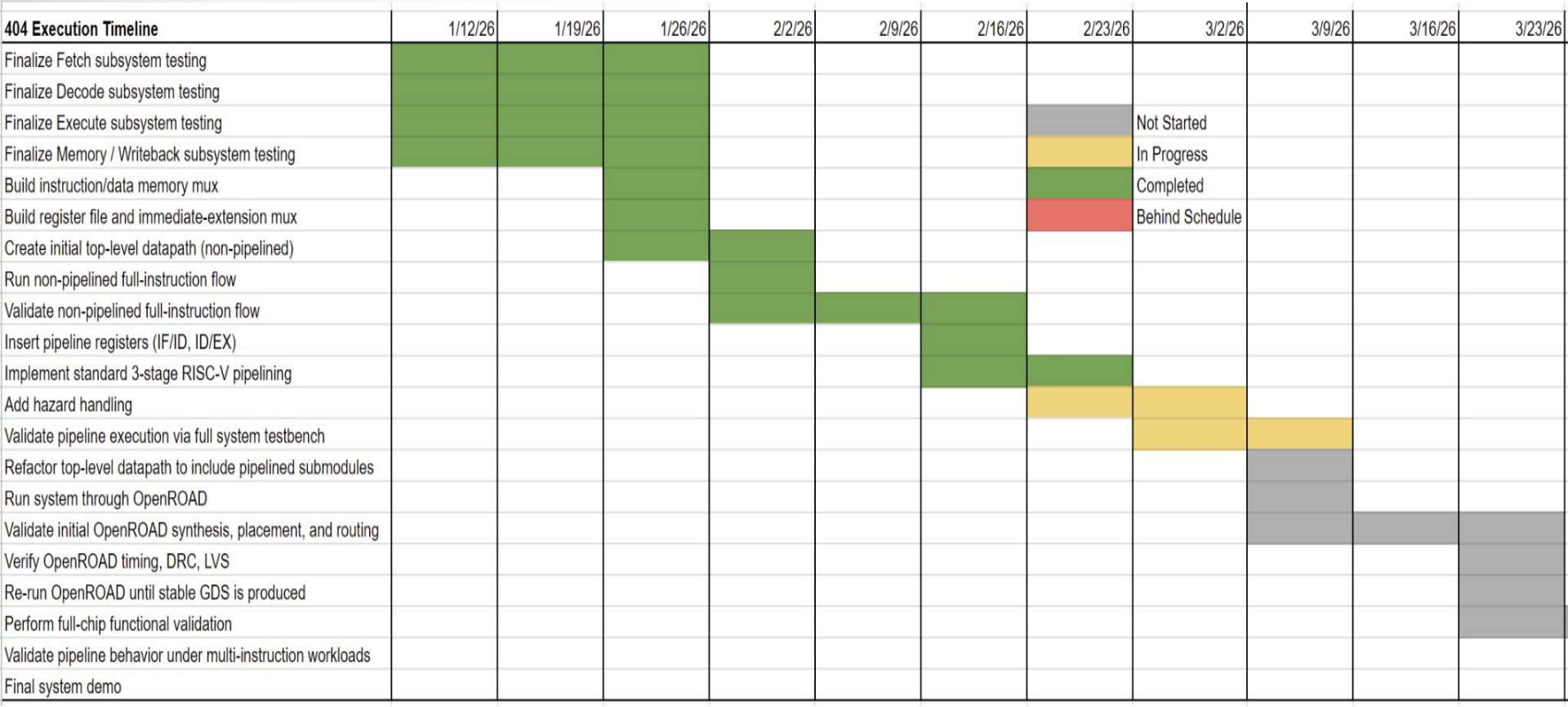
Jump/Branch Test Results

Giovani Giagnacovo & Githin Jonny

```
=== T12: JAL ===
1096000 | 0000 | 0000  | 0000 | 0001 | fffc | fff7
1106000 | 0000 | 0000  | 0000 | 0001 | fffc | fff7
1116000 | 0000 | 0000  | 0000 | 0001 | fffc | fff7
1126000 | 0000 | 008d  | 0000 | 0001 | fffc | fff7
1136000 | 0000 | 0000  | 0002 | 0001 | fffc | fff7
1146000 | 0002 | 0000  | 0000 | 0002 | fffc | fff7
1156000 | 0000 | 0000  | 0000 | 0002 | fffc | fff7
1166000 | 0000 | 8581  | 0000 | 0002 | fffc | fff7
1176000 | 0004 | 0000  | fff3 | 0002 | fffc | fff7
1186000 | 0006 | 0000  | 0000 | 0002 | fffc | fff3
1196000 | 0008 | 0000  | 0000 | 0002 | fffc | fff3
1206000 | 000a | 0000  | 0000 | 0002 | fffc | fff3
  FAIL  x3=33 (JAL landing)                  got=0xfff3  exp=0x0021
  PASS  x1=2  (JAL return addr)              got=0x0002

=== T13: JALR ===
1216000 | 0000 | 0000  | 0000 | 0002 | fffc | fff3
1226000 | 0000 | 0000  | 0000 | 0002 | fffc | fff3
1236000 | 0000 | 0000  | 0000 | 0002 | fffc | fff3
1246000 | 0000 | 1881  | 0000 | 0002 | fffc | fff3
1256000 | 0000 | 0000  | 0002 | 0002 | fffc | fff3
1266000 | 0002 | 0057  | 0000 | 0002 | fffc | fff3
1276000 | 0004 | 6181  | fffc | 0002 | fffc | fff3
1286000 | 0006 | 0000  | fff6 | 0002 | 0006 | fff3
1296000 | 0000 | 0000  | 0000 | 0002 | 0006 | fff6
1306000 | 0000 | 0000  | 0000 | 0002 | 0006 | fff6
1316000 | fffc | 0000  | 0000 | 0002 | 0006 | fff6
1326000 | fffe | 1881  | 0000 | 0002 | 0006 | fff6
  FAIL  x3=88 (JALR landing)                 got=0xfff6  exp=0x0058
  PASS  x2=6  (JALR ret addr)                got=0x0006
```

Execution Plan



Validation Plan

Paragraph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
3.1	Tile Size Constraint	The final GDSII layout must remain within the 2-tile maximum area allowed for TinyTapeout.	Run final Place and Route in OpenROAD, verify bounding box and padframe placement using the layout area report.	Untested	Full Team
4.1	DRC	Zero DRC violations (including antenna) in Magic and KLayout using the Sky130A/TinyTapeout rule decks.	Run Magic and KLayout DRC on the final GDS and iterate fixes until zero violations.	Untested	Full Team
4.2	Operating Temperature Range	Design must remain functionally valid across process and temperature corners (-55°C to +125°C).	Run timing and power analysis using PDK corner simulations (TT, SS, FF) to ensure performance remains stable within range.	Untested	Talha Ibrahim, Kevin Umanzor
5.1	LVS	Netgen reports “LVS clean” with 0 opens/shorts and all devices/nets/pins matched.	Extract the layout to SPICE and compare against the synthesized gate-level netlist in Netgen with the sky130A setup.	Untested	Full Team
5.2	Primary Input Power	Active power below 200 µW and lower during idle conditions.	Estimate and verify power through post-layout power analysis and simulation reports in OpenROAD.	Untested	Talha Ibrahim, Kevin Umanzor
5.3	Signal Interfaces	All interfaces (I²C, SPI, USB) respond correctly at specified logic levels and timings.	Run RTL and gate-level testbenches to verify correct data transfer, logic levels, and signal timing for all interfaces.	Untested	Githin Johny, Giovani Giagnacovo
5.4	User Control Interface	The reset mechanism returns the CPU to a known initial state.	Run RTL simulations for both synchronous and asynchronous resets to confirm the PC and register file reset correctly.	Untested	Giovani Giagnacovo, Talha Ibrahim
5.5	Clock Interface	The design accepts a 10 MHz external clock and meets all setup and hold timing constraints.	Run Static Timing Analysis (STA) and post-layout timing simulations in OpenROAD to verify clock stability and timing closure.	Untested	Kevin Umanzor, Githin Johny
6.1	Fetch	The Fetch stage retrieves correct instructions and updates the program counter properly.	Create directed and random test programs in RTL simulation to verify instruction fetch and PC increment behavior.	Tested	Kevin Umanzor
6.2	Decode	The Decode stage correctly generates control signals from instruction fields.	Run RTL testbenches for all supported instructions to confirm control signal accuracy and complete coverage.	Tested	Githin Johny
6.3	Execute	The ALU produces correct results for all arithmetic and logic operations.	Run RTL and gate-level simulations for arithmetic, logical, and branch operations with directed and random test vectors.	Tested	Talha Ibrahim
6.4	Memory/Write Back	The memory and write-back stages perform correct data reads, writes, and register updates.	Run RTL and gate-level simulations with memory models to verify correct load/store behavior and data integrity.	Tested	Giovani Giagnacovo
	Full System Simulation	The full 16-bit RISC-V design executes test programs correctly through all pipeline stages.	Integrate all modules into a top-level RTL and post-layout simulation, verifying instruction execution, output correctness, and timing.	Untested	Full Team
	Full System Demo	Execute a test program on each corresponding part on the chip mesuring correct output for all cases; along with measured power and basic I/O pass	Integrate tests for top level regeression and postlayout simulation captruing outputs.	Untested	Full Team

Thank You!